



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB REPORT

Title: OWASP Top 10 Risk Assessment and Secure Application Design for a Vulnerable Web Platform

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course: Web Application Security Essentials

Course Code: BVWS102

Instructor: AHMED BUKAR

1/DEC/2025

SECTION 1

1.0

Encryption means turning readable data into unreadable data using a key, so only authorized parties can turn it back into readable form. It protects data in transit and at rest.

TYPES OF ENCRYPTION

- Symmetric encryption (one key): The same key encrypts and decrypts. It's fast and used for large data (e.g., AES). The challenge: securely sharing the key.
- Asymmetric encryption (two keys): A public key encrypts; a private key decrypts. It enables secure key exchange and digital signatures (e.g., RSA, ECC). Slower, often used to exchange symmetric keys.
- Hashing (one-way): Converts data to a fixed-length digest that cannot be reversed (e.g., SHA-256). Used for integrity checks and storing passwords with salt + slow hashing.

IMPORTANCE OF ENCRYPTION FOR WEB APPS

ENCRYPTION TYPE	IMPORTANCE
SYMMETRIC	• Efficient for encrypting session data or files • Forms the bulk of HTTPS once a session is established
ASYMMETRIC	• Powers TLS handshakes • Certificate validation • Secure key exchange without exposing secrets
HASHING	• Secures passwords • Verifies data integrity • Prevents stolen password databases from revealing real passwords

2.0

Symmetric Encryption (one key for lock/unlock): Think of a shared locker at the gym. You and your friend both have the same key. Whoever locks it can unlock it with that same key. This is like encrypting data with AES fast and efficient, but both sides need the same secret key.

Asymmetric Encryption (two different keys): Imagine a mailbox at your house. Anyone can drop letters in through the slot (public key), but only you with the mailbox key (private key) can open it and read them. This is how RSA/ECC works the website gives out its public key, but only the server's private key can unlock the information.

Hashing (one-way fingerprint): Think of a recipe turned into a cake. Once baked, you can't turn the cake back into the recipe. But if someone else bakes the same recipe, you can compare cakes to see if they match. Passwords are hashed the system doesn't store the recipe (password), only the "cake" (hash). When you log in, it checks if your password produces the same "cake."

SECTION 2

1.0

HOW SSL/TLS WORKS

This is what happens when a user visits a secure website. The browser and the server use SSL/TLS to make sure the connection is private and trustworthy. It's like setting up a safe channel before any information is exchanged.

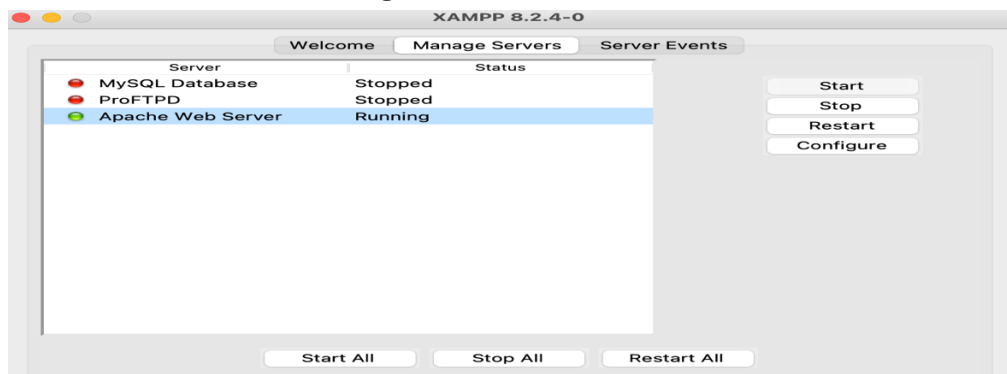
During the handshake, the browser and the server agree on how they will communicate securely. The browser says what security methods it supports, the server picks one, and they exchange information to create a shared secret key. Once that key is set, all the data sent between them is encrypted so nobody else can read it.

The certificate works like the website's ID card. It proves the site is real and not a fake copy. Certificates are issued by trusted companies called Certificate Authorities (CAs). The browser checks the certificate before trusting the site, and if it's valid, the secure connection continues.

The lock icon in the browser shows up when the connection is secure. It means the site has a valid certificate and that all the information between you and the site is encrypted. When you see the lock, you know your passwords, messages, or payments are being sent safely and can't easily be stolen or changed.

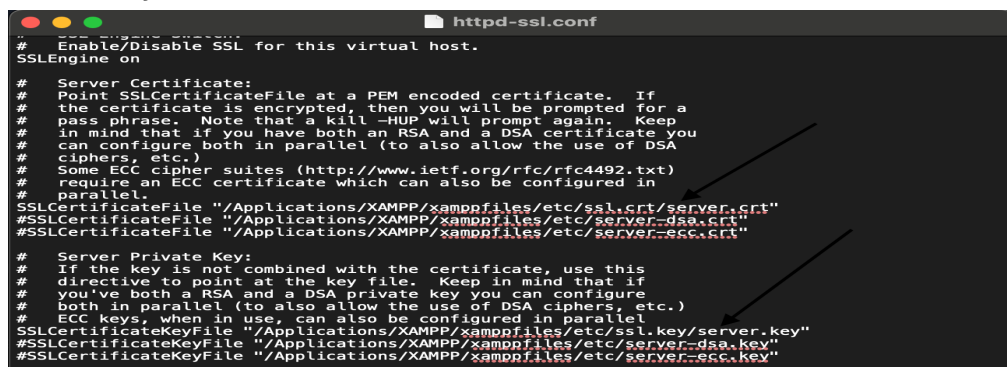
2.0

1. Opened XAMPP Control Panel → went to Manage Servers → selected Apache Web Server → clicked Configure.



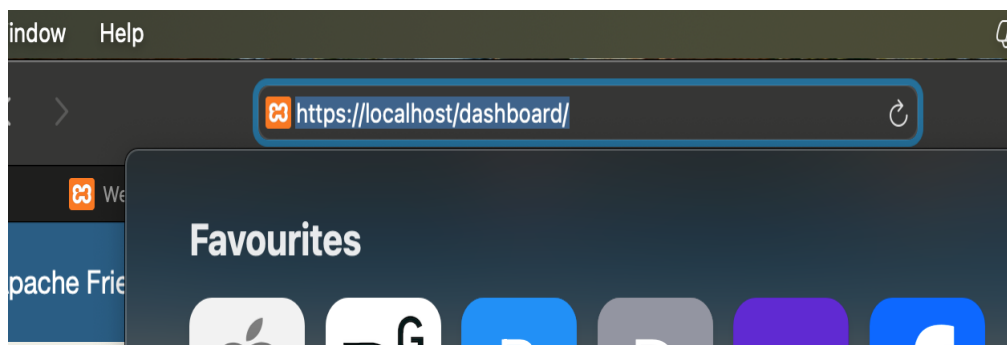
2. Found the file httpd-ssl.conf.

3. Saved the certificate and key into the XAMPP etc folder, then updated httpd-ssl.conf to point to them SSLCertificateFile "/Applications/XAMPP/xamppfiles/etc/server.crt" SSLCertificateKeyFile "/Applications/XAMPP/xamppfiles/etc/server.key".

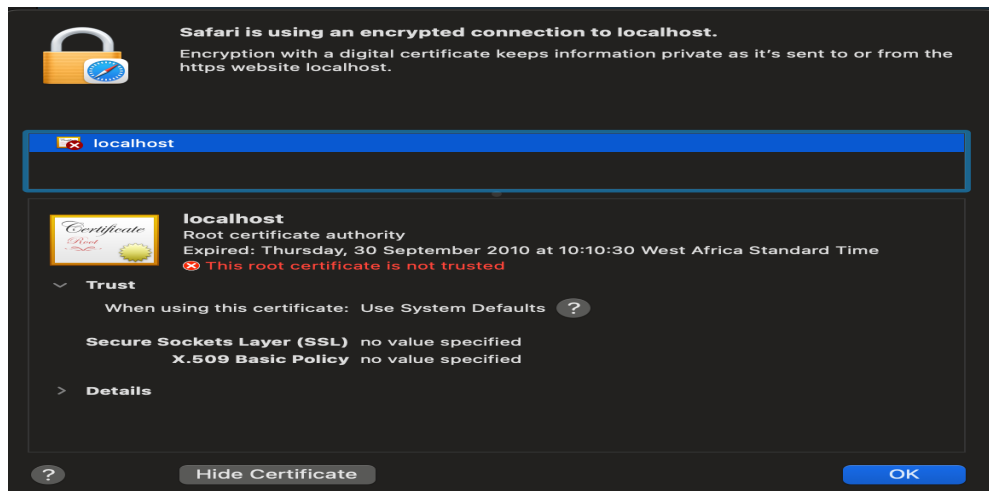


4. Restarted Apache from the control panel.

5. Tested in browser with <https://localhost>.



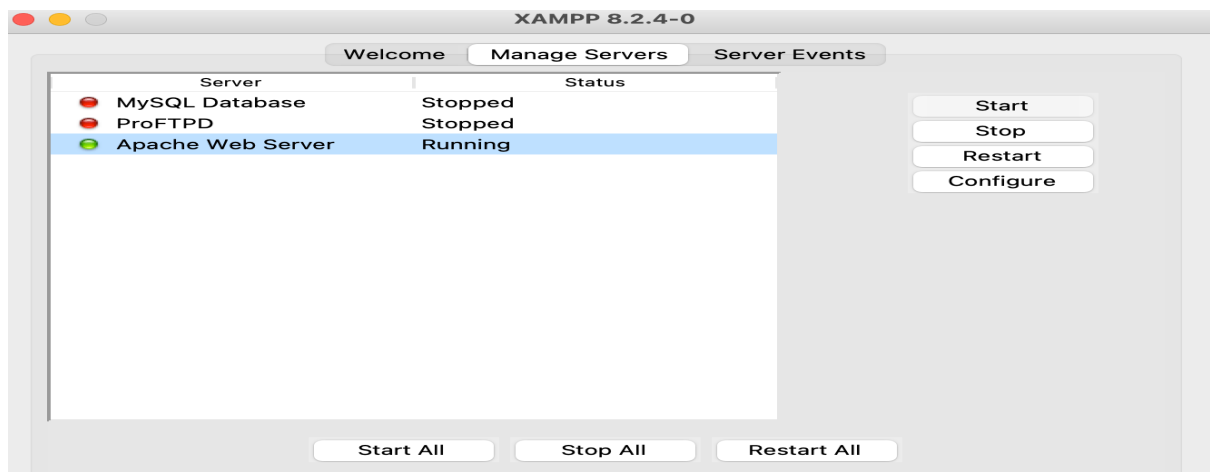
6. Safari showed a warning because the certificate was self-signed and not from a trusted Certificate Authority. I had to choose to continue before the site loaded. Safari no longer shows the padlock icon because Apple removed it in recent updates; instead, you verify security by clicking the site information menu in the address bar, which displays connection status and certificate details.



SECTION 3

DISABLE UNNECESSARY SERVICES

WHAT I DID: I stopped and disabled extra XAMPP services like ProFTPD that I didn't need.



This reduces the attack surface, making it harder for attackers to exploit unused services.

CONFIGURE A FIREWALL

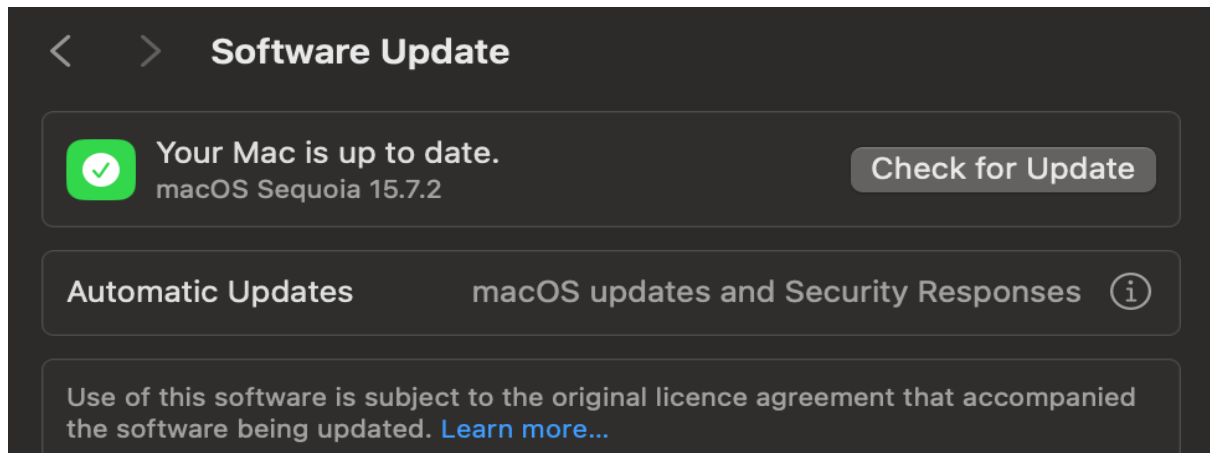
WHAT I DID: I enabled the macOS firewall and allowed only Apache while blocking other unnecessary connections.



This prevents unauthorized access from outside and ensures only trusted traffic reaches the server.

APPLY SYSTEM UPDATES

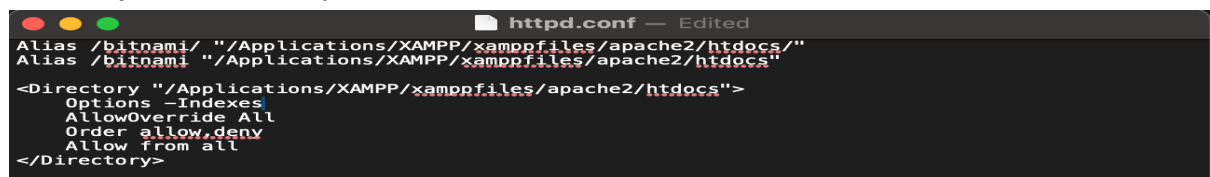
WHAT I DID: I updated macOS and XAMPP components to the latest versions.



This prevents attackers from exploiting known vulnerabilities and keeps the system secure with patches.

DISABLE DIRECTORY LISTING

I edited Apache's config file (httpd.conf) and changed "Options Indexes FollowSymLinks" to "Options -Indexes"



then restarted Apache. This stopped the browser from showing all files in a folder when no index file is present.

Forbidden

You don't have permission to access this resource.

It matters because it prevents attackers from browsing sensitive directories and discovering hidden files.

REMOVE DEFAULT CREDENTIALS

I opened phpMyAdmin and checked the user accounts. I found three root accounts (root@localhost, root@127.0.0.1, and root@::1) that had no password set. I edited the root@localhost account's privileges and assigned a password "garba", then updated the config.inc.php file in phpMyAdmin to match the new password so the connection would work properly.

```

/* Authentication type */
$cfg['Servers'][$i]['auth_type'] = 'cookie';
$cfg['Servers'][$i]['user'] = 'root';
$cfg['Servers'][$i]['password'] = 'garba';
/* Server parameters */
//$cfg['Servers'][$i]['host'] = 'localhost';

```

Finally, I switched the authentication type from 'config' to 'cookie' so phpMyAdmin would prompt me to enter the password at login instead of logging in automatically.

Why this matters because default or empty credentials are one of the easiest ways attackers gain access to databases. By setting strong passwords for all root accounts and requiring manual login, I closed a major security loophole, ensured only authorized users can access MySQL, and added an extra layer of protection against unauthorized access.

SECTION 4

Password Hashing

To protect user passwords, I used bcrypt to hash them before saving to the database: “php \$hashedPassword = password_hash(\$password, PASSWORD_BCRYPT)”. This ensures that even if the database is compromised, attackers cannot read the actual passwords.

	id	username	password_hash
☐ Edit Copy Delete	6	icdfa	\$2y\$10\$92IXUNpkjO0rOQ5byMi.Y
☐ Edit Copy Delete	7	admin	\$2y\$10\$92IXUNpkjO0rOQ5byMi.Y

phpMyAdmin showing two users with secured using bcrypt hash for passwords stored in the `users` table.

Prepared Statements

To prevent SQL injection, I used prepared statements that separate user input from the SQL query: `php $stmt = $mysqli->prepare("SELECT password_hash FROM users WHERE username = ?") $stmt->bind_param("s", $username).`

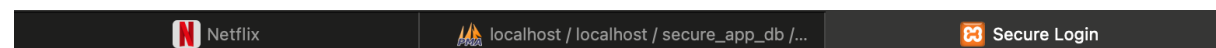
This blocks attackers from injecting malicious SQL through the login form.

Secure Error Messages

Instead of exposing system errors to users, I log them internally and show a generic message: `php error_log("Prepare error: " . $mysqli->error); exit("Something went wrong. Please try again.");` This keeps attackers from learning how the system works while still helping developers debug.

Input Validation

To clean user input and block harmful data, I sanitized the username like this: `php $username = filter_var($username, FILTER_SANITIZE_STRING);` This prevents cross-site scripting (XSS) and other injection attacks by removing unsafe characters.



Login

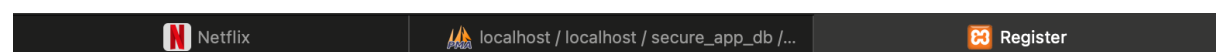
Username

Password

Login

[Create an account](#)

Shows a basic login form with fields for username and password, plus a link to register.



Register

Username

Password (min 8 chars, mix letters & numbers)

Create Account

Displays a simple registration form with password strength requirements for secure account creation.

Login
Invalid login.

Username

Password

[Create an account](#)

Displays an error when credentials are incorrect, confirming secure login handling.

Absolutely, Ahmed — here's a professional, straightforward, and original-sounding one-page summary based entirely on your full project journey from this chat:

SECTION 5

FINAL PROJECT SUMMARY

This project is a secure web-based login and registration system built using HTML, PHP, and MySQL. The application allows users to create accounts and log in through a simple interface, while implementing multiple layers of security to protect user data and system integrity.

The login and registration forms were designed with clear input fields and validation rules. Passwords are hashed using bcrypt before being stored in the database, ensuring that even if the database is compromised, the actual passwords remain protected. The application uses prepared statements for all database queries, which prevents SQL injection by separating user input from SQL logic.

To further enhance security, input sanitization was applied to all user-submitted data. This helps prevent cross-site scripting (XSS) and other injection attacks. Error handling was configured to hide system-level messages from users while logging them internally for debugging. This prevents attackers from gaining insights into the system's structure.

The application runs over HTTPS, which encrypts all communication between the browser and server. This protects sensitive data like login credentials from being intercepted during transmission. Server hardening was also considered by restricting access to configuration files and using secure database credentials.

If this system were deployed in a real company, I would improve it by adding multi-factor authentication, enforcing stronger password policies, and implementing rate limiting to prevent brute-force attacks. I would also move sensitive credentials to environment variables and conduct regular security audits.

Overall, this project demonstrates a practical and secure approach to user authentication, combining simplicity with essential protections to ensure safe and reliable access control.